```
// Best Fit

#include <stdio.h>

int main() {

    int m, n;

    // ==============================
    // INPUT NUMBER OF MEMORY BLOCKS
    // ==============================

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    /*
    Example:

    m = 5 blocks

    Blocks:
    B1, B2, B3, B4, B5
    */

    // ==============================
    // DECLARING BLOCK ARRAYS
    // ==============================

    int blockSize[m];   // stores remaining size of each block
    int original[m];    // stores original size for display

    // ==============================
    // INPUT BLOCK SIZES
    // ==============================

    printf("Enter size of each block:\n");

    /*
    Example:

    Block Sizes:
    100 500 200 300 600
    */
```

```
for(int i = 0; i < m; i++) {
    scanf("%d", &blockSize[i]);

    /*
    Save original block size
    because blockSize[] will change
    after allocation
    */

    original[i] = blockSize[i];
}

// ==============================
// INPUT NUMBER OF PROCESSES
// ==============================

printf("Enter number of processes:\n");
scanf("%d", &n);

/*
Example:

n = 4 processes

Processes:
P1, P2, P3, P4
*/

// ==============================
// DECLARING PROCESS ARRAY
// ==============================

int processSize[n];

// ==============================
// INPUT PROCESS SIZES
// ==============================

printf("Enter size of each process:\n");

/*
Example:

Process Sizes:
```

```
212 417 112 426
*/

for(int i = 0; i < n; i++) {
    scanf("%d", &processSize[i]);
}

// ==============================
// BEST FIT ALGORITHM STARTS
// ==============================

/*
Best Fit Strategy:

- For each process:
  → Check ALL memory blocks
  → Choose the block with
    MINIMUM leftover space

- This reduces wastage
*/

for(int i = 0; i < n; i++) {

    int bestIndex = -1;

    /*
    bestIndex stores the index
    of the best suitable block

    -1 means no block found yet
    */

    // ==============================
    // SEARCH ALL BLOCKS
    // ==============================

    for(int j = 0; j < m; j++) {

        /*
        Check if block can fit process
        */

        if(blockSize[j] >= processSize[i]) {
```

```
        /*
        Select block if:
        1. No block selected yet
        OR
        2. Current block is smaller
           than previously selected block
        */

        if(bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {
            bestIndex = j;
        }
    }
}

// ==============================
// IF BEST BLOCK FOUND
// ==============================

if(bestIndex != -1) {

    /*
    Internal Fragmentation:
    leftover space after allocation
    */

    int fragment = blockSize[bestIndex] - processSize[i];

    // ==============================
    // PRINT ALLOCATION DETAILS
    // ==============================

    printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
           i + 1, processSize[i], bestIndex + 1,
           original[bestIndex], fragment);

    /*
    Reduce block size after allocation
    (remaining memory reused)
    */

    blockSize[bestIndex] -= processSize[i];
}
```

```
        // ==============================
        // IF NO BLOCK FOUND
        // ==============================

        else {

            printf("Process %d of size %d is not allocated\n",
                i + 1, processSize[i]);
        }
    }

    return 0;
}
// First Fit

#include <stdio.h>

int main() {

    int m, n;

    // ==============================
    // INPUT NUMBER OF MEMORY BLOCKS
    // ==============================

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    /*
    Example:

    m = 5 blocks

    Blocks:
    B1, B2, B3, B4, B5
    */

    // ==============================
    // DECLARING BLOCK ARRAYS
    // ==============================

    int blockSize[m];          // stores current (remaining) size of each block
    int originalBlockSize[m];  // stores original size for display purpose
```

```
// ==============================
// INPUT BLOCK SIZES
// ==============================

printf("Enter size of each block:\n");

/*
Example:

Block Sizes:
100 500 200 300 600
*/

for(int i = 0; i < m; i++) {
    scanf("%d", &blockSize[i]);

    /*
    Store original size separately
    because blockSize[] will change
    after allocation (splitting)
    */

    originalBlockSize[i] = blockSize[i];
}

// ==============================
// INPUT NUMBER OF PROCESSES
// ==============================

printf("Enter number of processes:\n");
scanf("%d", &n);

/*
Example:

n = 4 processes

Processes:
P1, P2, P3, P4
*/

// ==============================
// DECLARING PROCESS ARRAY
// ==============================
```

```
int processSize[n];

// ==============================
// INPUT PROCESS SIZES
// ==============================

printf("Enter size of each process:\n");

/*
Example:

Process Sizes:
212 417 112 426
*/

for(int i = 0; i < n; i++) {
    scanf("%d", &processSize[i]);
}

// ==============================
// FIRST FIT ALGORITHM STARTS
// ==============================

/*
First Fit Strategy:

- For each process, scan blocks from beginning
- Allocate the FIRST block that is large enough
- Reduce the block size (memory splitting)
*/

for(int i = 0; i < n; i++) {

    int allocated = 0;

    /*
    allocated = 1 → process allocated
    allocated = 0 → process not allocated
    */

    // ==============================
    // SEARCH BLOCKS SEQUENTIALLY
    // ==============================
```

```
        for(int j = 0; j < m; j++) {

            /*
            Condition to allocate:
            block must have enough space
            */

            if(blockSize[j] >= processSize[i]) {

                /*
                Internal Fragmentation:
                leftover space in that block
                */

                int fragment = blockSize[j] - processSize[i];

                // ==============================
                // PRINT ALLOCATION DETAILS
                // ==============================

                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment
%d\n",
                       i + 1, processSize[i], j + 1,
                       originalBlockSize[j], fragment);

                /*
                Reduce block size after allocation
                (remaining memory can be reused)
                */

                blockSize[j] -= processSize[i];

                allocated = 1;

                /*
                Break because First Fit uses
                the FIRST suitable block only
                */

                break;
            }
        }
```

```
        // ==============================
        // IF NO BLOCK FOUND
        // ==============================

        if(!allocated) {

            printf("Process %d of size %d is not allocated\n",
                  i + 1, processSize[i]);
        }
    }

    return 0;
}
```